

Section 87 Editorial — Master Matching And Matroids Mixed Contest

A. Bottleneck Assignment

How to find it

Binary-search the threshold at which a perfect bipartite matching first exists.

Correctness

Feasibility is monotone and equivalent to an assignment with maximum cost at most the threshold.

Complexity

The reference follows the standard polynomial bound of the named augmentation, flow, or contraction algorithm; explicit problem limits keep oracle-based matroid intersection practical.

There are logarithmically many threshold decisions, each one bipartite matching. Feasibility is monotone because raising the threshold only adds edges; without that sentence, binary search has not been justified.

C++

```
#include <bits/stdc++.h>
using namespace std;
bool ok(vector<vector<int>> &a, int x) {
    int n = a.size();
    vector<int> match(n, -1);
    function<bool(int, vector<char> &)> dfs = [&](int u, vector<char> &seen) {
        for (int v = 0; v < n; v++)
            if (a[u][v] <= x && !seen[v]) {
                seen[v] = 1;
                if (match[v] < 0 || dfs(match[v], seen)) {
                    match[v] = u;
                    return true;
                }
            }
        return false;
    };
    for (int u = 0; u < n; u++) {
        vector<char> seen(n);
        if (!dfs(u, seen))
            return false;
    }
    return true;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<vector<int>> a(n, vector<int>(n));
    vector<int> v;
```

```

for (auto &r : a)
    for (int &x : r)
        cin >> x, v.push_back(x);
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
int l = -1, r = v.size() - 1;
while (r - l > 1) {
    int m = (l + r) / 2;
    (ok(a, v[m]) ? r : l) = m;
}
cout << v[r] << '\n';
}

```

Python

```

import sys

def possible(a, x):
    n = len(a)
    match = [-1] * n

    def dfs(u, seen):
        for v in range(n):
            if a[u][v] <= x and not seen[v]:
                seen[v] = 1
                if match[v] < 0 or dfs(match[v], seen):
                    match[v] = u
                    return True
        return False

    return all(dfs(u, [False] * n) for u in range(n))

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n = d[0]
    a = [d[1 + i * n : 1 + (i + 1) * n] for i in range(n)]
    values = sorted(set(d[1:]))
    l = -1
    r = len(values) - 1
    while r - l > 1:
        m = (l + r) // 2
        if possible(a, values[m]):
            r = m
        else:
            l = m
    print(values[r])

if __name__ == "__main__":
    main()

```

B. Roommate Rescue

How to find it

Use a maximum matching in the compatibility graph.

Correctness

Each matched edge covers exactly two participants, so maximum edge count minimizes the unmatched remainder.

Complexity

The reference follows the standard polynomial bound of the named augmentation, flow, or contraction algorithm; explicit problem limits keep oracle-based matroid intersection practical.

General matching takes $O(n^3)$ in the reference. The final number is $n - 2 * \text{matching_size}$; maximizing the number of edges and maximizing covered vertices are equivalent because matching edges are disjoint.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
pair<ll, vector<int>> hungarian(vector<vector<ll>> a) {
    int n = a.size(), m = a[0].size();
    vector<ll> u(n + 1), v(m + 1);
    vector<int> p(m + 1), way(m + 1);
    for (int i = 1; i <= n; i++) {
        p[0] = i;
        int j0 = 0;
        vector<ll> mn(m + 1, 4e18);
        vector<char> used(m + 1);
        do {
            used[j0] = 1;
            int i0 = p[j0], j1 = 0;
            ll delta = 4e18;
            for (int j = 1; j <= m; j++)
                if (!used[j]) {
                    ll cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                    if (cur < mn[j])
                        mn[j] = cur, way[j] = j0;
                    if (mn[j] < delta)
                        delta = mn[j], j1 = j;
                }
            for (int j = 0; j <= m; j++)
                if (used[j])
                    u[p[j]] += delta, v[j] -= delta;
                else
                    mn[j] -= delta;
            j0 = j1;
        } while (p[j0]);
        do {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }
    vector<int> match(n);
    for (int j = 1; j <= m; j++)
        if (p[j])
            match[p[j] - 1] = j - 1;
    return {-v[0], match};
}

struct Blossom {
    int n;
    vector<vector<int>> g;
    vector<int> match, p, base, q;
    vector<char> used, flower;
```

```

Blossom(vector<vector<int>>> g)
    : n(g.size()), g(g), match(n, -1), p(n), base(n), q(n), used(n),
      flower(n) {}
int lca(int a, int b) {
    vector<char> mark(n);
    while (1) {
        a = base[a];
        mark[a] = 1;
        if (match[a] < 0)
            break;
        a = p[match[a]];
    }
    while (1) {
        b = base[b];
        if (mark[b])
            return b;
        b = p[match[b]];
    }
}
void mark_path(int v, int b, int child) {
    while (base[v] != b) {
        flower[base[v]] = flower[base[match[v]]] = 1;
        p[v] = child;
        child = match[v];
        v = p[match[v]];
    }
}
int path(int root) {
    fill(used.begin(), used.end(), 0);
    fill(p.begin(), p.end(), -1);
    iota(base.begin(), base.end(), 0);
    int qh = 0, qt = 0;
    q[qt++] = root;
    used[root] = 1;
    while (qh < qt) {
        int v = q[qh++];
        for (int u : g[v])
            if (base[v] != base[u] && match[v] != u) {
                if (u == root || (match[u] >= 0 && p[match[u]] >= 0)) {
                    int b = lca(v, u);
                    fill(flower.begin(), flower.end(), 0);
                    mark_path(v, b, u);
                    mark_path(u, b, v);
                    for (int x = 0; x < n; x++)
                        if (flower[base[x]]) {
                            base[x] = b;
                            if (!used[x])
                                used[x] = 1, q[qt++] = x;
                        }
                }
                else if (p[u] < 0) {
                    p[u] = v;
                    if (match[u] < 0)
                        return u;
                    u = match[u];
                    used[u] = 1;
                    q[qt++] = u;
                }
            }
    }
    return -1;
}
int solve() {
    for (int root = 0; root < n; root++)

```

```

    if (match[root] < 0) {
        int v = path(root);
        while (v >= 0) {
            int pv = p[v], nv = pv < 0 ? -1 : match[pv];
            match[v] = pv;
            if (pv >= 0)
                match[pv] = v;
            v = nv;
        }
    }

    return count_if(match.begin(), match.end(), [](int x) { return x >= 0; }) /
        2;
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> g(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    cout << n - 2 * Blossom(g).solve() << '\n';
}

```

Python

```

import sys

def hungarian(cost):
    n = len(cost)
    m = len(cost[0])
    u = [0] * (n + 1)
    v = [0] * (m + 1)
    p = [0] * (m + 1)
    way = [0] * (m + 1)
    for i in range(1, n + 1):
        p[0] = i
        j0 = 0
        minimum = [10**30] * (m + 1)
        used = [False] * (m + 1)
        while True:
            used[j0] = True
            i0 = p[j0]
            delta = 10**30
            j1 = 0
            for j in range(1, m + 1):
                if not used[j]:
                    cur = cost[i0 - 1][j - 1] - u[i0] - v[j]
                    if cur < minimum[j]:
                        minimum[j] = cur
                        way[j] = j0
            if minimum[j] < delta:
                delta = minimum[j]

```

```

        j1 = j
    for j in range(m + 1):
        if used[j]:
            u[p[j]] += delta
            v[j] -= delta
        else:
            minimum[j] -= delta
    j0 = j1
    if p[j0] == 0:
        break
    while True:
        j1 = way[j0]
        p[j0] = p[j1]
        j0 = j1
        if j0 == 0:
            break
    match = [-1] * n
    for j in range(1, m + 1):
        if p[j]:
            match[p[j] - 1] = j - 1
    return -v[0], match

def blossom(graph):
    n = len(graph)
    match = [-1] * n
    base = list(range(n))
    parent = [-1] * n

    def find_path(root):
        nonlocal base, parent
        used = [False] * n
        parent = [-1] * n
        base = list(range(n))
        q = [root]
        used[root] = True

    def get_lca(a, b):
        mark = [False] * n
        while True:
            a = base[a]
            mark[a] = True
            if match[a] == -1:
                break
            a = parent[match[a]]
        while True:
            b = base[b]
            if mark[b]:
                return b
            b = parent[match[b]]

    def mark_path(v, b, child, flower):
        while base[v] != b:
            flower[base[match[v]]] = flower[base[match[v]]] = True
            parent[v] = child
            child = match[v]
            v = parent[match[v]]

    for v in q:
        for u in graph[v]:
            if base[v] == base[u] or match[v] == u:
                continue
            if u == root or (match[u] != -1 and parent[match[u]] != -1):

```

```

        b = get_lca(v, u)
        flower = [False] * n
        mark_path(v, b, u, flower)
        mark_path(u, b, v, flower)
        for x in range(n):
            if flower[base[x]]:
                base[x] = b
            if not used[x] and flower[base[x]]:
                used[x] = True
                q.append(x)
        elif parent[u] == -1:
            parent[u] = v
            if match[u] == -1:
                return u
            u = match[u]
            used[u] = True
            q.append(u)
    return -1

for root in range(n):
    if match[root] != -1:
        continue
    v = find_path(root)
    if v == -1:
        continue
    while v != -1:
        pv = parent[v]
        nv = match[pv] if pv != -1 else -1
        match[v] = pv
        if pv != -1:
            match[pv] = v
        v = nv
    return match

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, m = d[:2]
    g = [[] for _ in range(n)]
    for i in range(2, 2 + 2 * m, 2):
        u, v = d[i] - 1, d[i + 1] - 1
        g[u].append(v)
        g[v].append(u)
    matched = sum(x >= 0 for x in blossom(g)) // 2
    print(n - 2 * matched)

if __name__ == "__main__":
    main()

```

C. Cut Threshold Pairs

How to find it

Enumerate and sort all Gomory–Hu path minima.

Correctness

Each stored value is exactly one unordered pair cut, so upper bound answers every threshold count.

Complexity

The reference follows the standard polynomial bound of the named augmentation, flow, or contraction algorithm; explicit problem limits keep oracle-based matroid intersection practical.

After the $n-1$ flows used to build the cut tree, the reference extracts all pair path minima in $O(n^3)$ and answers each threshold by binary search. This is appropriate for the stated size; larger limits would call for offline tree-edge aggregation rather than changing the cut-tree theorem.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Dinic {
    struct E {
        int v, rev;
        ll c;
    };
    int n;
    vector<vector<E>> g;
    vector<int> level, it;
    vector<char> side;
    Dinic(int n) : n(n), g(n), level(n), it(n), side(n) {}
    void add(int u, int v, ll c) {
        g[u].push_back({v, (int)g[v].size(), c});
        g[v].push_back({u, (int)g[u].size() - 1, 0});
    }
    ll dfs(int u, int t, ll f) {
        if (u == t)
            return f;
        for (int &i = it[u]; i < (int)g[u].size(); i++) {
            E &e = g[u][i];
            if (e.c && level[e.v] == level[u] + 1) {
                ll z = dfs(e.v, t, min(f, e.c));
                if (z) {
                    e.c -= z;
                    g[e.v][e.rev].c += z;
                    return z;
                }
            }
        }
        return 0;
    }
    ll flow(int s, int t) {
        ll ans = 0;
        while (1) {
            fill(level.begin(), level.end(), -1);
            queue<int> q;
            q.push(s);
            level[s] = 0;
            while (!q.empty()) {
                int u = q.front();
                q.pop();
                for (auto &e : g[u])
                    if (e.c && level[e.v] < 0)
                        level[e.v] = level[u] + 1, q.push(e.v);
            }
            if (level[t] < 0) {
                for (int i = 0; i < n; i++)
                    side[i] = level[i] >= 0;
                return ans;
            }
        }
    }
};
```

```

    }
    fill(it.begin(), it.end(), 0);
    while (ll z = dfs(s, t, 4e18))
        ans += z;
}
}
};

vector<vector<pair<int, ll>>> gomory(int n, vector<tuple<int, int, ll>> e) {
    vector<int> p(n);
    vector<ll> w(n);
    for (int s = 1; s < n; s++) {
        int t = p[s];
        Dinic d(n);
        for (auto [u, v, c] : e)
            d.add(u, v, c), d.add(v, u, c);
        w[s] = d.flow(s, t);
        for (int v = s + 1; v < n; v++)
            if (p[v] == t && d.side[v])
                p[v] = s;
        if (d.side[p[t]])
            p[s] = p[t], p[t] = s, swap(w[s], w[t]);
    }
    vector<vector<pair<int, ll>>> tree(n);
    for (int v = 1; v < n; v++)
        tree[v].push_back({p[v], w[v]}), tree[p[v]].push_back({v, w[v]});
    return tree;
}

struct MinTree {
    int n, L;
    vector<int> dep;
    vector<vector<int>> up;
    vector<vector<ll>> mn;
    MinTree(vector<vector<pair<int, ll>>> t)
        : n(t.size()), L(max(1, (int)bit_width((unsigned)n))), dep(n),
          up(L, vector<int>(n)), mn(L, vector<ll>(n, 4e18)) {
        vector<pair<int, int>> st = {{0, 0}};
        while (!st.empty()) {
            auto [u, p] = st.back();
            st.pop_back();
            up[0][u] = p;
            for (auto [v, w] : t[u])
                if (v != p)
                    dep[v] = dep[u] + 1, mn[0][v] = w, st.push_back({v, u});
        }
        for (int j = 1; j < L; j++)
            for (int v = 0; v < n; v++)
                mn[j][v] = min(mn[j - 1][v], mn[j - 1][up[j - 1][v]]),
                up[j][v] = up[j - 1][up[j - 1][v]];
    }
    ll query(int a, int b) {
        ll ans = 4e18;
        if (dep[a] < dep[b])
            swap(a, b);
        int z = dep[a] - dep[b];
        for (int j = 0; j < L; j++)
            if (z >> j & 1)
                ans = min(ans, mn[j][a]), a = up[j][a];
        if (a == b)
            return ans;
        for (int j = L - 1; j >= 0; j--)
            if (up[j][a] != up[j][b])
                ans = min({ans, mn[j][a], mn[j][b]}), a = up[j][a], b = up[j][b];
        return min({ans, mn[0][a], mn[0][b]});
    }
};

```

```

    }
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    vector<tuple<int, int, ll>> e(m);
    for (auto &[u, v, c] : e)
        cin >> u >> v >> c, --u, --v;
    MinTree t(gomory(n, e));
    vector<ll> a;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            a.push_back(t.query(i, j));
    sort(a.begin(), a.end());
    while (q--) {
        ll x;
        cin >> x;
        cout << upper_bound(a.begin(), a.end(), x) - a.begin() << '\n';
    }
}

```

Python

```

import sys
from collections import deque

class Dinic:
    def __init__(self, n):
        self.g = [[] for _ in range(n)]

    def add(self, u, v, c):
        self.g[u].append([v, c, len(self.g[v])])
        self.g[v].append([u, 0, len(self.g[u]) - 1])

    def flow(self, s, t):
        answer = 0
        n = len(self.g)
        while True:
            level = [-1] * n
            level[s] = 0
            q = deque([s])
            while q:
                u = q.popleft()
                for v, c, _ in self.g[u]:
                    if c and level[v] < 0:
                        level[v] = level[u] + 1
                        q.append(v)
            if level[t] < 0:
                self.reachable = [x >= 0 for x in level]
                return answer
            it = [0] * n

            def dfs(u, pushed):
                if u == t:
                    return pushed
                while it[u] < len(self.g[u]):
                    e = self.g[u][it[u]]
                    if e[1] and level[e[0]] == level[u] + 1:

```

```

        take = dfs(e[0], min(pushed, e[1]))
        if take:
            e[1] -= take
            self.g[e[0]][e[2]][1] += take
            return take
        it[u] += 1
    return 0

    while True:
        pushed = dfs(s, 10**30)
        if not pushed:
            break
        answer += pushed

def gomory_hu(n, edges):
    parent = [0] * n
    value = [0] * n
    for s in range(1, n):
        t = parent[s]
        d = Dinic(n)
        for u, v, c in edges:
            d.add(u, v, c)
            d.add(v, u, c)
        value[s] = d.flow(s, t)
        side = d.reachable
        for v in range(s + 1, n):
            if parent[v] == t and side[v]:
                parent[v] = s
        if side[parent[t]]:
            parent[s] = parent[t]
            parent[t] = s
            value[s], value[t] = value[t], value[s]
    tree = [[] for _ in range(n)]
    for v in range(1, n):
        tree[v].append((parent[v], value[v]))
        tree[parent[v]].append((v, value[v]))
    return tree

def tree_minimum(tree):
    n = len(tree)
    L = max(1, n.bit_length())
    up = [[0] * n for _ in range(L)]
    mn = [[10**30] * n for _ in range(L)]
    depth = [0] * n
    stack = [(0, 0)]
    while stack:
        u, p = stack.pop()
        up[0][u] = p
        for v, w in tree[u]:
            if v != p:
                depth[v] = depth[u] + 1
                mn[0][v] = w
                stack.append((v, u))
    for j in range(1, L):
        for v in range(n):
            mn[j][v] = min(mn[j - 1][v], mn[j - 1][up[j - 1][v]])
            up[j][v] = up[j - 1][up[j - 1][v]]

    def query(a, b):
        answer = 10**30
        if depth[a] < depth[b]:

```

```

        a, b = b, a
        z = depth[a] - depth[b]
        for j in range(L):
            if z >> j & 1:
                answer = min(answer, mn[j][a])
                a = up[j][a]
        if a == b:
            return answer
        for j in range(L - 1, -1, -1):
            if up[j][a] != up[j][b]:
                answer = min(answer, mn[j][a], mn[j][b])
                a, b = up[j][a], up[j][b]
        return min(answer, mn[0][a], mn[0][b])

    return query

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, m, q = d[:3]
    at = 3
    e = []
    for _ in range(m):
        u, v, c = d[at : at + 3]
        at += 3
        e.append((u - 1, v - 1, c))
    query = tree_minimum(gomory_hu(n, e))
    values = sorted(query(i, j) for i in range(n) for j in range(i))
    import bisect

    print(
        "\n".join(
            str(bisect.bisect_right(values, d[i])) for i in range(at, at + q)
        )
    )

if __name__ == "__main__":
    main()

```

D. Rainbow Spanning Tree

How to find it

Seek common independent size $n-1$ in graphic and color matroids.

Correctness

A forest of $n-1$ edges is spanning, making the size test necessary and sufficient.

Complexity

The reference follows the standard polynomial bound of the named augmentation, flow, or contraction algorithm; explicit problem limits keep oracle-based matroid intersection practical.

The target size matters: graphic independence only promises a forest, and partition independence only promises distinct colors. Reaching exactly $n-1$ turns the forest into a spanning tree and completes the equivalence.

C++

```
#include <bits/stdc++.h>
using namespace std;
struct DSU {
    vector<int> p;
    DSU(int n) : p(n) { iota(p.begin(), p.end(), 0); }
    int f(int x) { return p[x] == x ? x : p[x] = f(p[x]); }
    bool add(int a, int b) {
        a = f(a);
        b = f(b);
        if (a == b)
            return 0;
        p[a] = b;
        return 1;
    }
};

bool forest_ok(int n, const vector<pair<int, int>> &e, const vector<char> &in) {
    DSU d(n);
    for (int i = 0; i < (int)e.size(); i++)
        if (in[i] && !d.add(e[i].first, e[i].second))
            return 0;
    return 1;
}

vector<char> intersect(int m, function<bool(const vector<char> &)> ok1,
                      function<bool(const vector<char> &)> ok2) {
    vector<char> in(m);
    while (1) {
        vector<int> par(m, -2), q;
        for (int e = 0; e < m; e++)
            if (!in[e]) {
                in[e] = 1;
                bool ok = ok1(in);
                in[e] = 0;
                if (ok)
                    par[e] = -1, q.push_back(e);
            }
        int finish = -1;
        for (int h = 0; h < (int)q.size() && finish < 0; h++) {
            int x = q[h];
            if (!in[x]) {
                in[x] = 1;
                bool sink = ok2(in);
                in[x] = 0;
                if (sink) {
                    finish = x;
                    break;
                }
            }
            for (int y = 0; y < m; y++)
                if (in[y] && par[y] == -2) {
                    in[y] = 0;
                    in[x] = 1;
                    bool ok = ok2(in);
                    in[x] = 0;
                    in[y] = 1;
                    if (ok)
                        par[y] = x, q.push_back(y);
                }
        }
        if (finish == -1)
            for (int e = 0; e < m; e++)
                if (!in[e] && par[e] == -2) {
                    in[x] = 0;
                    in[e] = 1;
                }
    }
}
```

```

        bool ok = ok1(in);
        in[e] = 0;
        in[x] = 1;
        if (ok)
            par[e] = x, q.push_back(e);
    }
}
if (finish < 0)
    return in;
for (int x = finish; x >= 0; x = par[x])
    in[x] ^= 1;
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> e(m);
    vector<int> color(m);
    for (int i = 0; i < m; i++)
        cin >> e[i].first >> e[i].second >> color[i], --e[i].first, --e[i].second;
    auto in = intersect(
        m, [&](auto &s) { return forest_ok(n, e, s); },
        [&](auto &s) {
            set<int> z;
            for (int i = 0; i < m; i++)
                if (s[i] && !z.insert(color[i]).second)
                    return false;
            return true;
        });
    cout << (count(in.begin(), in.end(), 1) == n - 1 ? "YES\n" : "NO\n");
}

```

Python

```

import sys

def forest_ok(n, edges, chosen):
    p = list(range(n))

    def find(x):
        while p[x] != x:
            p[x] = p[p[x]]
            x = p[x]
        return x

    for i in chosen:
        u, v = edges[i]
        u = find(u)
        v = find(v)
        if u == v:
            return False
        p[u] = v
    return True

def partition_ok(colors, chosen):
    return len({colors[i] for i in chosen}) == len(chosen)

```

```

def intersection(m, ok1, ok2):
    inside = set()
    while True:
        outside = [e for e in range(m) if e not in inside]
        parent = {}
        q = []
        for e in outside:
            if ok1(inside | {e}):
                parent[e] = -1
                q.append(e)
        finish = -1
        for x in q:
            if x not in inside and ok2(inside | {x}):
                finish = x
                break
            if x in inside:
                for e in outside:
                    if e not in parent and ok1((inside - {x}) | {e}):
                        parent[e] = x
                        q.append(e)
                else:
                    for y in list(inside):
                        if y not in parent and ok2((inside - {y}) | {x}):
                            parent[y] = x
                            q.append(y)
        if finish < 0:
            return inside
        x = finish
        while x != -1:
            if x in inside:
                inside.remove(x)
            else:
                inside.add(x)
            x = parent[x]

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, m = d[:2]
    edges = []
    colors = []
    for i in range(2, 2 + 3 * m, 3):
        edges.append((d[i] - 1, d[i + 1] - 1))
        colors.append(d[i + 2])
    answer = intersection(
        m,
        lambda chosen: forest_ok(n, edges, chosen),
        lambda chosen: partition_ok(colors, chosen),
    )
    print("YES" if len(answer) == n - 1 else "NO")

if __name__ == "__main__":
    main()

```

E. Broadcast Backbone

How to find it

Apply directed minimum-arborescence contraction with signed reduced costs.

Correctness

Loops and parallel or negative edges do not alter the minimum-incoming/cycle replacement proof.

Complexity

The reference follows the standard polynomial bound of the named augmentation, flow, or contraction algorithm; explicit problem limits keep oracle-based matroid intersection practical.

The implementation is $O(VE)$. Negative weights are harmless because each contraction charges mandatory incoming minima and carries only replacement costs onward; reachability from the root remains a separate feasibility issue.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Edge {
    int u, v;
    ll w;
};
optional<ll> arbo(int n, int root, vector<Edge> e) {
    ll ans = 0;
    while (1) {
        vector<ll> in(n, 4e18);
        vector<int> pre(n, -1);
        for (auto x : e)
            if (x.u != x.v && x.w < in[x.v])
                in[x.v] = x.w, pre[x.v] = x.u;
        in[root] = 0;
        for (ll x : in)
            if (x == 4e18)
                return {};
        for (ll x : in)
            ans += x;
        vector<int> id(n, -1), seen(n, -1);
        int count = 0;
        for (int s = 0; s < n; s++) {
            int v = s;
            while (seen[v] != s && id[v] < 0 && v != root)
                seen[v] = s, v = pre[v];
            if (v != root && id[v] < 0) {
                for (int u = pre[v]; u != v; u = pre[u])
                    id[u] = count;
                id[v] = count++;
            }
        }
        if (!count)
            return ans;
        for (int v = 0; v < n; v++)
            if (id[v] < 0)
                id[v] = count++;
        vector<Edge> next;
        for (auto x : e)
            next.push_back({id[x.u], id[x.v], x.w - in[x.v]});
        root = id[root];
        n = count;
        e.swap(next);
    }
};
int main() {
    ios::sync_with_stdio(false);
```

```

cin.tie(nullptr);
int n, m, r;
cin >> n >> m >> r;
vector<Edge> e(m);
for (auto &x : e)
    cin >> x.u >> x.v >> x.w, --x.u, --x.v;
auto ans = arbo(n, r - 1, e);
if (ans)
    cout << *ans << '\n';
else
    cout << "IMPOSSIBLE\n";
}

```

Python

```

import sys

def arborescence(n, root, edges):
    answer = 0
    while True:
        incoming = [10**30] * n
        pre = [-1] * n
        for u, v, w in edges:
            if u != v and w < incoming[v]:
                incoming[v] = w
                pre[v] = u
        incoming[root] = 0
        if any(x == 10**30 for x in incoming):
            return None
        answer += sum(incoming)
        component = [-1] * n
        seen = [-1] * n
        count = 0
        for start in range(n):
            v = start
            while seen[v] != start and component[v] < 0 and v != root:
                seen[v] = start
                v = pre[v]
            if v != root and component[v] < 0:
                u = pre[v]
                component[v] = count
                while u != v:
                    component[u] = count
                    u = pre[u]
                count += 1
        if count == 0:
            return answer
        for v in range(n):
            if component[v] < 0:
                component[v] = count
                count += 1
        edges = [
            (component[u], component[v], w - incoming[v]) for u, v, w in edges
        ]
        root = component[root]
        n = count

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))

```

```

n, m, root = d[:3]
edges = []
for i in range(3, 3 + 3 * m, 3):
    edges.append((d[i] - 1, d[i + 1] - 1, d[i + 2]))
answer = arborescence(n, root - 1, edges)
print("IMPOSSIBLE" if answer is None else answer)

if __name__ == "__main__":
    main()

```

F. Two-Map Forest

How to find it

Seek size $n-1$ in two graphic matroids.

Correctness

That size is a spanning tree on both n -vertex maps, and every common spanning tree is such a common independent set.

Complexity

The reference follows the standard polynomial bound of the named augmentation, flow, or contraction algorithm; explicit problem limits keep oracle-based matroid intersection practical.

Both endpoint maps must have the same vertex count stated by the problem, but their connectivity evolves independently. A common independent set of size below $n-1$ is not a partial answer: it certifies that no common spanning tree exists only after matroid intersection has reached maximum cardinality.

C++

```

#include <bits/stdc++.h>
using namespace std;
struct DSU {
    vector<int> p;
    DSU(int n) : p(n) { iota(p.begin(), p.end(), 0); }
    int f(int x) { return p[x] == x ? x : p[x] = f(p[x]); }
    bool add(int a, int b) {
        a = f(a);
        b = f(b);
        if (a == b)
            return 0;
        p[a] = b;
        return 1;
    }
};
bool forest_ok(int n, const vector<pair<int, int>> &e, const vector<char> &in) {
    DSU d(n);
    for (int i = 0; i < (int)e.size(); i++)
        if (in[i] && !d.add(e[i].first, e[i].second))
            return 0;
    return 1;
}
vector<char> intersect(int m, function<bool(const vector<char> &)> ok1,
                        function<bool(const vector<char> &)> ok2) {
    vector<char> in(m);
    while (1) {
        vector<int> par(m, -2), q;

```

```

for (int e = 0; e < m; e++)
    if (!in[e]) {
        in[e] = 1;
        bool ok = ok1(in);
        in[e] = 0;
        if (ok)
            par[e] = -1, q.push_back(e);
    }
int finish = -1;
for (int h = 0; h < (int)q.size() && finish < 0; h++) {
    int x = q[h];
    if (!in[x]) {
        in[x] = 1;
        bool sink = ok2(in);
        in[x] = 0;
        if (sink) {
            finish = x;
            break;
        }
    }
    for (int y = 0; y < m; y++)
        if (in[y] && par[y] == -2) {
            in[y] = 0;
            in[x] = 1;
            bool ok = ok2(in);
            in[x] = 0;
            in[y] = 1;
            if (ok)
                par[y] = x, q.push_back(y);
        }
    } else
        for (int e = 0; e < m; e++)
            if (!in[e] && par[e] == -2) {
                in[x] = 0;
                in[e] = 1;
                bool ok = ok1(in);
                in[e] = 0;
                in[x] = 1;
                if (ok)
                    par[e] = x, q.push_back(e);
            }
    }
    if (finish < 0)
        return in;
    for (int x = finish; x >= 0; x = par[x])
        in[x] ^= 1;
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    int n1 = n, n2 = n;
    vector<pair<int, int>> a(m), b(m);
    for (int i = 0; i < m; i++)
        cin >> a[i].first >> a[i].second >> b[i].first >> b[i].second, --a[i].first,
            --a[i].second, --b[i].first, --b[i].second;
    auto in = intersect(
        m, [&](auto &s) { return forest_ok(n1, a, s); },
        [&](auto &s) { return forest_ok(n2, b, s); });
    cout << (count(in.begin(), in.end(), 1) == n - 1 ? "YES\n" : "NO\n");
}

```

Python

```
import sys

def forest_ok(n, edges, chosen):
    p = list(range(n))

    def find(x):
        while p[x] != x:
            p[x] = p[p[x]]
            x = p[x]
        return x

    for i in chosen:
        u, v = edges[i]
        u = find(u)
        v = find(v)
        if u == v:
            return False
        p[u] = v
    return True

def partition_ok(colors, chosen):
    return len({colors[i] for i in chosen}) == len(chosen)

def intersection(m, ok1, ok2):
    inside = set()
    while True:
        outside = [e for e in range(m) if e not in inside]
        parent = {}
        q = []
        for e in outside:
            if ok1(inside | {e}):
                parent[e] = -1
                q.append(e)
        finish = -1
        for x in q:
            if x not in inside and ok2(inside | {x}):
                finish = x
                break
            if x in inside:
                for e in outside:
                    if e not in parent and ok1((inside - {x}) | {e}):
                        parent[e] = x
                        q.append(e)
                else:
                    for y in list(inside):
                        if y not in parent and ok2((inside - {y}) | {x}):
                            parent[y] = x
                            q.append(y)
        if finish < 0:
            return inside
        x = finish
        while x != -1:
            if x in inside:
                inside.remove(x)
            else:
                inside.add(x)
            x = parent[x]
```

```

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, m = d[:2]
    n1 = n2 = n
    at = 2
    first = []
    second = []
    for i in range(m):
        a, b, c, e = d[at : at + 4]
        at += 4
        first.append((a - 1, b - 1))
        second.append((c - 1, e - 1))
    answer = intersection(
        m,
        lambda chosen: forest_ok(n1, first, chosen),
        lambda chosen: forest_ok(n2, second, chosen),
    )
    print("YES" if len(answer) == n - 1 else "NO")

if __name__ == "__main__":
    main()

```

Mixed-contest stress plan

Use permutation enumeration for A, recursive matching enumeration for B, pairwise max flow for C, subset enumeration for D and F, and incoming-edge enumeration for E. Generate both feasible and impossible targets. This set is particularly sensitive to reductions that are almost right, so compare the reported optimum as well as the validity of any reconstructed object.